

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)
Assessment Tool Weightage:40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply

8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze

Code of Conduct:

*I, Judy Allen J (192520055) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
Judy Allen J

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate	Applies concepts with minor errors	Limited application	Unable to apply

		concepts, methods, or tools		; requires guidance	relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justificatio n	No justification provided

Q1. Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.

Problem Understanding

The objective is to optimize a complex SQL query by reducing unnecessary tuple processing and intermediate results. The major constraints are the number of joins, selection conditions, projection of required attributes, and the order in which operations are performed.

Consider the following DBMS query:

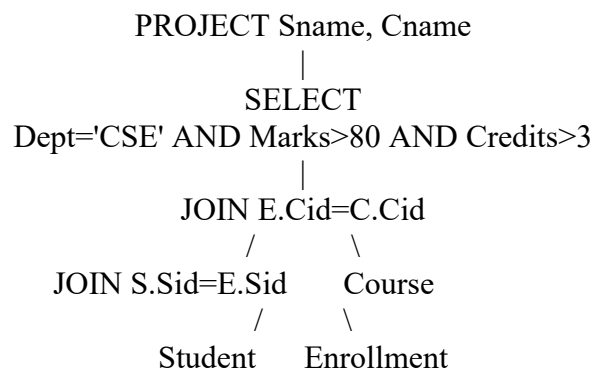
```
SELECT S.Sname, C.Cname
FROM Student S
JOIN Enrollment E ON S.Sid = E.Sid
JOIN Course C ON E.Cid = C.Cid
WHERE S.Dept = 'CSE'
AND E.Marks > 80
AND C.Credits > 3;
```

Constraint Analysis

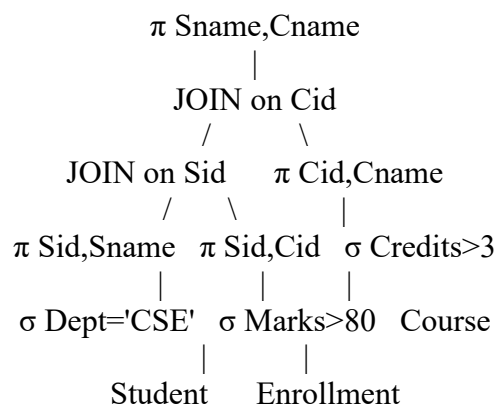
The main constraints are:

- Selection conditions should be applied as early as possible.
- Unnecessary attributes should be removed before joins.
- Smaller intermediate relations should be joined first.
- The optimized query must return the same result as the original query.

Initial Query Execution Plan



Optimized Execution Plan



Solution Development

Step 1: Push selection operations close to the corresponding tables.

$\sigma_{\text{Dept}=\text{'CSE'}}(\text{Student})$

$\sigma_{\text{Marks}>80}(\text{Enrollment})$

$\sigma_{\text{Credits}>3}(\text{Course})$

Step 2: Apply projection early.

$\pi_{\text{Sid}, \text{Sname}}(\text{Student})$

$\pi_{\text{Sid}, \text{Cid}}(\text{Enrollment})$

$\pi_{\text{Cid}, \text{Cname}}(\text{Course})$

Step 3: Join the reduced Student and Enrollment relations.

$\text{Student_Result} \bowtie \text{Enrollment_Result}$

Step 4: Join the result with the reduced Course relation.

Step 5: Perform the final projection.

$\pi_{\text{Sname}, \text{Cname}}$

Application of DBMS Concepts

The solution applies heuristic query optimization techniques such as selection push-down, projection push-down, and efficient join ordering.

Justification

Applying selection and projection before join operations reduces the number and size of tuples processed. Therefore, disk I/O, memory usage, and execution time are reduced while preserving the correctness of the original query.

Q2. Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join.

Problem Understanding

Cost-based optimization compares different query execution methods and selects the one with the lowest estimated cost.

Assume:

Relation R = 10,000 tuples, 100 pages

Relation S = 5,000 tuples, 50 pages

Available buffer pages = 12

Constraint Analysis

The optimizer must consider:

- Number of pages in each relation.
- Available memory buffers.
- Disk I/O cost.
- Type of join condition.
- Size of intermediate relations.

Solution Development

Nested-Loop Join

Using S as the outer relation:

$$\text{Cost} = B(S) + B(S) \times B(R)$$

$$\begin{aligned}\text{Cost} &= 50 + (50 \times 100) \\ &= 5050 \text{ page accesses}\end{aligned}$$

Sort-Merge Join

Approximate sorting cost:

$$\text{Sort R} = 4 \times 100 = 400$$

$$\text{Sort S} = 4 \times 50 = 200$$

Merge cost:

$$100 + 50 = 150$$

Total cost:

$$400 + 200 + 150 = 750 \text{ page accesses}$$

Hash Join

$$\text{Cost} = 3 \times (B(R) + B(S))$$

$$\begin{aligned}\text{Cost} &= 3 \times (100 + 50) \\ &= 450 \text{ page accesses}\end{aligned}$$

Comparison

Join Method	Estimated Cost
Nested-Loop Join	5050 page accesses
Sort-Merge Join	750 page accesses
Hash Join	450 page accesses

Application of DBMS Concepts

Cost-based query optimization estimates the resource requirements of alternative execution plans and selects the least expensive plan.

Justification

Hash join requires approximately 450 page accesses, which is lower than sort-merge join and nested-loop join. Therefore, Hash Join is the optimal choice for the given equality join.

Q3. Implement JDBC connectivity in Java to a MySQL database and execute parameterized queries using Prepared Statement.

Problem Understanding

The objective is to connect a Java application to a MySQL database and securely execute a parameterized SQL query.

Constraint Analysis

The program must:

- Establish JDBC connectivity.
- Connect to the correct MySQL database.
- Use PreparedStatement.
- Accept parameter values dynamically.
- Handle database errors.
- Close database resources correctly.

Solution Development

```
import java.sql.*;
import java.util.Scanner;

public class JDBCExample {
    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/college";
        String user = "root";
        String password = "root";

        Scanner sc = new Scanner(System.in);

        try {
            Connection con =
                DriverManager.getConnection(url, user, password);

            System.out.println("Database Connected Successfully");

            String sql =
                "SELECT Sid, Sname, Dept FROM Student WHERE Dept = ?";

            PreparedStatement ps =
                con.prepareStatement(sql);

            System.out.print("Enter Department: ");
            String dept = sc.nextLine();

            ps.setString(1, dept);

            ResultSet rs = ps.executeQuery();

            while (rs.next()) {
                System.out.println(
```



```

        rs.getInt("Sid") + " " +
        rs.getString("Sname") + " " +
        rs.getString("Dept")
    );
}

rs.close();
ps.close();
con.close();

} catch (SQLException e) {
    System.out.println("Database Error: " + e.getMessage());
}

sc.close();
}
}

```

Application of DBMS Concepts

The program uses JDBC components such as:

- Connection
- PreparedStatement
- ResultSet
- DriverManager

The placeholder ? is used in the SQL query and the parameter is assigned using:

```
ps.setString(1, dept);
```

Justification

PreparedStatement is preferred because it separates SQL commands from user input. It improves security by reducing SQL injection risks and is efficient when the same query is executed multiple times.

Q4. Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.

Problem Understanding

Conflict serializability is determined by identifying conflicting operations between transactions and constructing a precedence graph.

Consider the schedule:

R1(A)

W1(A)

R2(A)

W2(B)

R3(B)

W3(C)

R4(C)

W4(D)

Constraint Analysis

Two operations conflict when:

- They belong to different transactions.
- They access the same data item.
- At least one operation is a write operation.

Solution Development

For data item A:

W1(A) occurs before R2(A)

Therefore:

$T1 \rightarrow T2$

For data item B:

W2(B) occurs before R3(B)

Therefore:

$T2 \rightarrow T3$

For data item C:

W3(C) occurs before R4(C)

Therefore:

$T3 \rightarrow T4$

Precedence Graph

$T1 \longrightarrow T2 \longrightarrow T3 \longrightarrow T4$

Application of DBMS Concepts

The precedence graph contains transactions as nodes and conflicting dependencies as directed edges.

There is no cycle in the graph.

Therefore, the schedule is conflict-serializable.

Equivalent serial order:

$T1 \rightarrow T2 \rightarrow T3 \rightarrow T4$

Justification

A schedule is conflict-serializable when its precedence graph is acyclic. Since the graph has no cycle, the given schedule is equivalent to the serial execution order T1, T2, T3, T4.

Q5. Demonstrate Basic Two-Phase Locking on a transaction set and identify and resolve deadlocks.

Problem Understanding

Basic Two-Phase Locking divides transaction execution into a growing phase and a shrinking phase.

Consider:

T1: Read(A), Write(B)

T2: Read(B), Write(A)

Constraint Analysis

Under Basic 2PL:

- Locks can be acquired during the growing phase.
- Once a lock is released, no new locks can be acquired.
- Conflicting exclusive locks cannot be granted simultaneously.

Solution Development

Initially:

T1 obtains X-lock(A)

T2 obtains X-lock(B)

T1 requests:

X-lock(B)

Since B is locked by T2:

T1 waits for T2

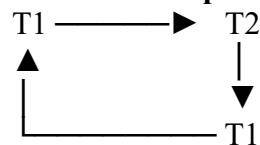
T2 requests:

X-lock(A)

Since A is locked by T1:

T2 waits for T1

Wait-For Graph



The graph contains a cycle. Hence, a deadlock exists.

Deadlock Resolution

Select T2 as the victim.

Abort T2

Release locks held by T2

Allow T1 to complete

Restart T2 after T1 commits

Correct execution:

T1:

Lock-X(A)

Lock-X(B)

Read(A)

Write(B)

Commit

Unlock(A)

Unlock(B)

T2:

Lock-X(B)

Lock-X(A)

Read(B)

Write(A)

Commit

Unlock(B)

Unlock(A)

Application of DBMS Concepts

Basic 2PL guarantees conflict serializability but can lead to deadlocks. A wait-for graph is used to detect circular waiting.

Justification

Aborting one transaction breaks the cycle in the wait-for graph. T1 can complete and release its locks, after which T2 can restart safely. Thus, database consistency is preserved.

Q6. Apply Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes.

Problem Understanding

Wait-Die and Wound-Wait are timestamp-based techniques used to prevent deadlocks.

Assume:

$TS(T1) = 10$

$TS(T2) = 20$

$TS(T3) = 30$

Therefore:

T1 = Oldest

T2 = Younger than T1

T3 = Youngest

Constraint Analysis

A smaller timestamp represents an older transaction. Transactions retain their original timestamps after restart to reduce starvation.

Solution Development

Wait-Die Scheme

Rules:

Older transaction requests younger transaction's lock → Wait

Younger transaction requests older transaction's lock → Abort

Example 1:

T1 requests a lock held by T2.

T1 is older than T2.

Result:

T1 waits.

Example 2:

T3 requests a lock held by T1.

T3 is younger than T1.

Result:

T3 is aborted.

Wound-Wait Scheme

Rules:

Older transaction requests younger transaction's lock → Abort younger transaction

Younger transaction requests older transaction's lock → Wait

Example 1:

T1 requests a lock held by T2.

T1 is older.

Result:

T2 is aborted.

T1 obtains the lock.

Example 2:

T3 requests a lock held by T1.

T3 is younger.

Result:

T3 waits.

Comparison

Situation	Wait-Die	Wound-Wait
Older requests younger's lock	Wait	Abort younger
Younger requests older's lock	Abort younger	Wait

Application of DBMS Concepts

Both techniques use transaction timestamps to maintain a consistent ordering of lock requests.

Justification

Since transactions are forced to follow timestamp ordering, circular waiting cannot occur. Therefore, both Wait-Die and Wound-Wait prevent deadlocks before they form.

Q7. Apply Immediate Update recovery protocol using transaction logs and checkpoints.

Problem Understanding

In Immediate Update, database changes may be written before the corresponding transaction commits. Therefore, recovery may require both UNDO and REDO operations.

Consider:

<T1, Start>

<T1, A, 100, 150>

<T1, Commit>

<Checkpoint>

<T2, Start>

<T2, B, 200, 250>

<T3, Start>

<T3, C, 300, 350>

<T3, Commit>

<System Crash>

Constraint Analysis

After the crash:

- Committed transactions must be preserved.
- Uncommitted transactions must not affect the database.
- Recovery begins from the latest checkpoint.

Solution Development

Transaction status:

T1 = Committed before checkpoint

T2 = Uncommitted

T3 = Committed after checkpoint

For T3:

REDO T3

C = 350

For T2:

UNDO T2

B = 200

T1 normally requires no action because it committed before the checkpoint.

Final Recovery

UNDO T2

REDO T3

Final values:

B = 200

C = 350

Application of DBMS Concepts

Immediate Update follows the UNDO/REDO recovery mechanism.

UNDO restores the old value of an uncommitted transaction.

REDO restores the new value of a committed transaction.

Justification

T2 must be undone because it did not commit before the crash. T3 must be redone because it committed and its updates must remain durable. This satisfies the ACID properties of atomicity and durability.

Q8. Apply Deferred Update NO-UNDO/REDO recovery technique.

Problem Understanding

In Deferred Update, changes made by a transaction are not permanently written to the database until the transaction commits.

Consider:

<T1, Start>

<T1, A, 100, 150>

<T1, Commit>

<T2, Start>

<T2, B, 200, 250>

<T3, Start>

<T3, C, 300, 400>

<T3, Commit>

<System Crash>

Constraint Analysis

The recovery system must:

- REDO committed transactions.
- Ignore uncommitted transactions.
- Avoid unnecessary UNDO operations.

Solution Development

Transaction status:

T1 = Committed

T2 = Uncommitted

T3 = Committed

Recovery:

REDO T1

A = 150

T2 is ignored because it did not commit.

NO UNDO for T2

Redo T3:

REDO T3

C = 400

Final Recovery

REDO T1

IGNORE T2

REDO T3

Application of DBMS Concepts

Deferred Update uses:

NO-UNDO / REDO

because uncommitted modifications are never permanently written to the database.

Justification

T2 does not need to be undone because its changes were never permanently stored. T1 and T3 are redone to guarantee durability of committed transactions. Therefore, Deferred Update simplifies the recovery process.

Q9. Design a concurrency control mechanism using Strict 2PL for a banking system.

Problem Understanding

Consider two banking transactions:

T1 = Transfer ₹1000 from Account A to Account B

T2 = Read the current balance of Account A

The system must ensure that T2 does not read a partially updated value while T1 is executing.

Constraint Analysis

The mechanism must prevent:

- Dirty reads.
- Lost updates.
- Inconsistent account balances.
- Cascading rollbacks.
- Reading uncommitted data.

Solution Development

Transaction T1:

X-Lock(A)

X-Lock(B)

Read(A)

$A = A - 1000$

Write(A)

Read(B)

$B = B + 1000$

Write(B)

Commit

Unlock(A)

Unlock(B)

Transaction T2:

S-Lock(A)

Read(A)

Commit

Unlock(A)

Execution

T1	T2
X-Lock(A)	
X-Lock(B)	
Read(A)	
Write(A)	
	S-Lock(A)
	WAIT
Read(B)	
Write(B)	
COMMIT	
Unlock(A)	
Unlock(B)	
	S-Lock(A)
	Read(A)
	COMMIT
	Unlock(A)

Application of DBMS Concepts

Under Strict 2PL, T1 retains its exclusive locks until commit. Therefore, T2 cannot obtain a shared lock on Account A while T1 has an exclusive lock.

Justification

Strict 2PL ensures that T2 reads the balance only after the money transfer has been successfully committed. This prevents dirty reads and cascading rollback and maintains the consistency and reliability required in a banking DBMS.

Q10. Write pseudocode for query optimization using heuristics.

Problem Understanding

The goal is to optimize SQL query execution using heuristic rules such as:

- Push down selection.
- Project attributes early.
- Join smaller intermediate relations first.

Constraint Analysis

The optimizer must:

- Preserve the result of the original SQL query.
- Reduce the number of tuples processed.
- Reduce tuple width.
- Minimize intermediate results.
- Reduce disk I/O and execution time.

Solution Development

Algorithm Heuristic_Query_Optimization(Query Q)

Input:

SQL Query Q

Output:

Optimized Query Execution Plan

Begin

1. Parse the SQL query Q.
2. Construct the initial query tree.
3. Identify all selection conditions.
4. For each selection condition:
Move the selection operation
close to its corresponding base relation.
5. Identify only the attributes required for:
SELECT clause,
WHERE clause,
JOIN conditions.
6. Apply projection on each relation
to remove unnecessary attributes.
7. Estimate the size of each filtered relation.
8. Arrange relations according to
increasing relation size.

9. Select the two smallest relations.
10. Perform their join first.
11. Store the result as an intermediate relation.
12. Join the intermediate relation with the next smallest relation.
13. Repeat until all required relations are joined.
14. Apply final projection.
15. Return the optimized query plan.

End

Example

Original query:

```
SELECT S.Sname, C.Cname
FROM Student S, Enrollment E, Course C
WHERE S.Sid = E.Sid
AND E.Cid = C.Cid
AND S.Dept = 'CSE'
AND E.Marks > 80;
```

Push selections:

$\sigma_{\text{Dept}='CSE'}(\text{Student})$

$\sigma_{\text{Marks}>80}(\text{Enrollment})$

Apply projections:

$\pi_{\text{Sid}, \text{Sname}}(\text{Student})$

$\pi_{\text{Sid}, \text{Cid}}(\text{Enrollment})$

$\pi_{\text{Cid}, \text{Cname}}(\text{Course})$

Perform smaller joins first:

Result1 =

$\text{Student} \bowtie \text{Enrollment}$

Then:

Result2 =

$\text{Result1} \bowtie \text{Course}$

Final operation:

$\pi_{\text{Sname}, \text{Cname}}(\text{Result2})$

Application of DBMS Concepts

The solution applies relational algebra and heuristic query optimization techniques used by DBMS query optimizers.

Justification

Selection push-down reduces the number of rows, early projection reduces the number of columns, and optimized join ordering reduces intermediate relation sizes. Therefore, the optimized query requires less memory, disk I/O, and execution time while producing the same final result.